# STUDYNOTION: AN ONLINE EDUCATION PLATFORM

*A project submitted to Annamalai University*

*In partial fulfilment of the requirement*

*For the award of the degree of*

**BACHELOR OF COMPUTER SCIENCE**

*Submitted by*


**P. SHARUKAVI (REG NO: 24132111802522033)**

**A. VIJAYASANTHI (REG NO: 24132111802522039)**

**R. POORNIMA (REG NO: 24132111802522021)**


*Under the guidance of*


**Mr. D. SURESH KUMAR M.Sc., M.Phil., B.Ed., Ph.D.,**

**Head of the department,**

**Department of Computer Science,**

**BWDA Arts and Science College.**





**BWDA ARTS AND SCIENCE COLLEGE**

**( Affliated to Annamalai University)**

**KOLLIYANGUNAM, OCT – 2026**

# CERTIFICATE

**Mr. D. SURESH KUMAR M.Sc., M.Phil., B.Ed., Ph.D.,**

Head of the department,

Department of Computer Science,

BWDA Arts and Science College,

Kolliyangunam.

This is to certify that this major project work entitled " **STUDYNOTION: AN ONLINE EDUCATION PLATFORM "** is a bonafide record of the work and submitted by

**P. SHARUKAVI (REG NO: 24132111802522033)**

**A. VIJAYASANTHI (REG NO: 24132111802522039)**

**R. POORNIMA (REG NO: 24132111802522021)**

in partial fulfilment of the requirement for the award of the **BACHELOR OF SCIENCE IN COMPUTER SCIENCE** during V Semester of Annamalai University.

I certify that the project work is carried out by his/her as independent work under my supervision and guidance.

**Signature of Guide** **Signature of HOD**

Submitted for the Viva-Voce examination held on _____

**External Examiners**

1.

2.

## DECLARATION

I affirm that the project work the **" STUDYNOTION : AN ONLINE EDUCATION PLATFORM"** being submitted in partial fulfilment for the award of Bachelor of Science in Computer Science is the original work carried out my carried out my me. It has not formed the part of project work submitted for the award of any degree of diploma either in this or any other university.

**Signature of the Candidate**

**P. SHARUKAVI (REG NO: 24132111802522033)**
**A. VIJAYASANTHI (REG NO: 24132111802522039)**
**R. POORNIMA (REG NO: 24132111802522021)**

I certify that the declaration made above by the candidate is true.

**Signature of the guide**

**Mr. D. SURESH KUMAR M.Sc., M.Phil., B.Ed., Ph.D.,**

# STUDYNOTION: AN ONLINE EDUCATION PLATFORM


## ABSTRACT

StudyNotion is a full-stack ed-tech platform that lets instructors create and sell courses and lets students browse, enrol in, learn from, and rate them. It is built on the MERN stack: ReactJS with Redux on the front end, and Node.js with Express.js on the back end, communicating through RESTful APIs. MongoDB (via Mongoose) powers the live API, and a parallel, normalised MySQL schema with foreign-key constraints has been added to evaluate a relational alternative. Key features include JWT-based authentication, Bcrypt password hashing, role-based access for students and instructors, course and section management, progress tracking, ratings and reviews, a wishlist, and Cloudinary-based media storage. A Razorpay payment integration is retained in the codebase. The main objective is to bridge the gap between instructors and learners by providing one accessible platform where educational content can be published, discovered, and consumed from any location, using a transparent, self-hostable implementation built entirely with open-source technologies. On the back end, the Express.js server separates routes, controllers, models, and middleware, and protected routes are guarded by JWT verification and role-based checks so that instructor-only actions cannot be performed by students. The MySQL schema contains eleven tables covering users, courses, enrolments, reviews, and progress, and enforces referential integrity through foreign keys and unique constraints. The system was validated through unit, integration, system, and user acceptance testing. Future work includes an administrator dashboard, live classes, AI-based course recommendations, and completing the migration of the live API to MySQL. StudyNotion thus offers a complete learning marketplace.

# ACKNOWLEDGEMENT

I take this privilege to express a few words of guidance and respect to all those who helped in completion of the project.

I am bound to express my gratitude to our honourable Principal **Dr. V. SEGAR M.A., M.Phil., Ph.D.,** BWDA ARTS AND SCIENCE COLLEGE for providing all the required facilities for the achievement of my project.

I am sincere thanks to **Mr. D. SURESH KUMAR M.Sc., M.Phil., B.Ed., Ph.D.,** Head of the department, Department of Computer Science for his constant guidance and encouragement throughout my project work.

I am sincere thanks to project guide, **Mr. D. SURESH KUMAR M.Sc., M.Phil., B.Ed., Ph.D.,** Head of the department, Department of Computer Science for his constant guidance and encouragement throughout my project work.

I take an opportunity to express my thanks to the management and administration of BWDA ARTS AND SCIENCE COLLEGE for given this an opportunity to study in this college.

Words are boundless to express my thanks to our department staff members for their kind assistant, cooperation's and valuable suggestion given to me during the project.

Finally my special thanks to my parents and friends for their continued support and encouragement my acknowledgement is also due to many others, who have rendered their help in my occasions during my project work.

**P. SHARUKAVI**

**A. VIJAYASANTHI**

**R. POORNIMA**

# CONTENT


| CHAPTER NO | TITLE | PAGE NO: |
|---|---|---|
| 1. | **INTRODUCTION** | 1 |
| | 1.1 Overview | |
| | 1.2 Motivation | |
| | 1.3 Problem Statement | |
| | 1.4 Objectives | |
| | 1.5 Scope of the Project | |
| | 1.6 Organization of the Report | |
| | 1.7 Advantages of the Proposed System | |
| 2. | **LITERATURE SURVEY** | 5 |
| | 2.1 Introduction | |
| | 2.2 Study of Existing E-Learning Platforms | |
| | 2.3 Comparative Study | |
| | 2.4 Research Gap | |
| | 2.5 Summary | |
| 3. | **SYSTEM ANALYSIS** | 8 |
| | 3.1 Existing System | |
| | 3.2 Proposed System | |
| | 3.3 Feasibility Study | |
| | 3.4 Software Requirement Specification (SRS) | |
| | 3.5 Hardware and Software Requirements | |
| | 3.6 Interface Requirements | |
| | 3.7 Use Case Descriptions | |
| | 3.8 Assumptions and Constraints | |
| | 3.9 Software Development Life Cycle (SDLC) Model | |
| | 3.10 Risk Analysis | |
| 4. | **SYSTEM DESIGN** | 17 |

| | | |
|---|---|---|
| | 4.1 System Architecture | |
| | 4.2 Data Flow Diagrams (DFD) | |
| | 4.3 Entity-Relationship (ER) Diagram | |
| | 4.4 Database Architecture: A Dual-Database Design | |
| | 4.5 UML Diagrams | |
| | 4.6 Module Description | |
| | 4.7 Data Dictionary | |
| | 4.8 Design Constraints | |
| | 4.9 Design Patterns Used | |
| 5. | **TECHNOLOGY STACK** | 30 |
| | 5.1 Overview of the MERN Stack | |
| | 5.2 Front-End Technologies | |
| | 5.3 Back-End Technologies | |
| | 5.4 Development and Deployment Tools | |
| | 5.5 Justification for Technology Choices | |
| 6. | **IMPLEMENTATION** | 35 |
| | 6.1 Development Approach | |
| | 6.2 Backend Project Structure | |
| | 6.3 Authentication and Security Implementation | |
| | 6.4 Course Management Implementation | |
| | 6.5 Payment Integration | |
| | 6.6 Media Management Implementation | |
| | 6.7 Frontend State Management | |
| | 6.8 Course Content Rendering | |
| | 6.9 Wishlist and Progress Tracking Implementation | |
| | 6.10 Deployment | |
| | 6.11 Error Handling and Logging | |
| | 6.12 Coding Standards and Version Control | |
| | 6.13 Sample Controller Code – Rating and Review | |
| | 6.15 Challenges Faced and Solutions | |

| | | |
|---|---|---|
| 7. | **TESTING** | 42 |
| | 7.1 Testing Objectives | |
| | 7.2 Testing Strategies: Black-Box and White-Box Testing | |
| | 7.3 Types of Testing Performed | |
| | 7.4 Test Cases and Results | |
| | 7.5 Discussion of Testing Results | |
| | 7.6 Test Environment | |
| | 7.7 Security Testing | |
| 8. | **RESULTS AND DISCUSSION** | 46 |
| | 8.1 Output Screens | |
| | 8.2 Discussion | |
| | 8.3 System Maintenance and Support | |
| | 8.4 Performance Observations | |
| 9 | **FUTURE ENHANCEMENTS** | 49 |
| | 9.1 Administrator Dashboard | |
| | 9.2 Live Classes and Webinars | |
| | 9.3 AI-Based Course Recommendations | |
| | 9.4 Discussion Forums and Peer Interaction | |
| | 9.5 Mobile Application | |
| | 9.6 Adaptive Video Streaming | |
| | 9.7 Certification System | |
| | 9.8 Completing the MySQL Migration | |
| | 9.9 Scalability Enhancements | |
| 10. | **CONCLUSION** | 51 |
| 11. | **REFERENCES** | 52 |

# CHAPTER 1

# INTRODUCTION

## 1.1 Overview

The rapid growth of internet penetration and the increasing affordability of smart devices have fundamentally transformed the way education is delivered and consumed. Traditional classroom-based learning, while still relevant, is increasingly being supplemented and, in many cases, replaced by online learning platforms that offer flexibility, scalability, and accessibility to a much wider audience. E-learning platforms allow students to learn at their own pace, revisit content as many times as required, and access courses from instructors located anywhere in the world.

StudyNotion is one such online education platform, developed as a full-stack web application using the MERN stack (MongoDB, Express.js, React.js, and Node.js). It is designed to serve as a two-sided marketplace connecting instructors, who create and publish courses, with students, who browse, purchase, and consume that content. The platform incorporates modern web development practices, secure authentication mechanisms, real-time payment processing, and cloud-based media delivery to provide a production-grade learning experience.

This chapter introduces the motivation behind the project, states the problem being addressed, defines the objectives and scope of the system, and outlines the overall organisation of this report.

## 1.2 Motivation

The motivation for building StudyNotion stems from the observation that quality education is often locked behind geographical and institutional boundaries. Many capable instructors do not have an easy way to package and monetise their knowledge, while many students, particularly those in remote areas or with limited access to formal institutions, struggle to find structured, affordable, and verifiable course content.

Existing commercial platforms solve parts of this problem but are often closed-source, expensive to customise, or unsuitable for academic study of how a full production-grade ed-tech system is actually engineered end-to-end — from authentication and payments to media delivery and content rendering. Building StudyNotion from first principles using the MERN stack provided an

opportunity to gain hands-on, practical experience with the entire life cycle of a real-world web application: requirement analysis, system design, implementation, testing, deployment, and future scaling considerations.

### 1.3 Problem Statement

There is a growing need for an accessible, affordable, and scalable platform that allows instructors to create and manage educational content while giving students a seamless and interactive way to discover, purchase, and consume that content. Existing solutions used within many educational institutions are either outdated, non-interactive, or lack essential e-commerce features such as secure payment processing, content rating, and progress tracking.

The problem, therefore, is to design and develop a full-stack web-based ed-tech platform that:

- Allows instructors to create, update, and manage multi-section courses with rich media content.
- Allows students to browse, wishlist, purchase, and consume courses in a structured manner.
- Provides secure authentication and role-based access control for students, instructors, and (in future scope) administrators.
- Integrates a reliable payment gateway to handle course purchases.
- Efficiently stores and serves media content such as videos, images, and documents.
- Is built on a scalable, maintainable, and modern technology stack.

### 1.4 Objectives

The primary objectives of the StudyNotion project are as follows:

1. To design and implement a responsive front-end interface using ReactJS that provides an engaging and intuitive user experience for both students and instructors.
2. To develop a secure and robust back-end using Node.js and Express.js that exposes a well-structured set of RESTful APIs for authentication, course management, and payments.

3. To design a flexible and scalable database schema using MongoDB to store user data, course content, ratings, and reviews.
4. To implement secure user authentication and authorization using JSON Web Tokens (JWT), OTP verification, and password hashing with Bcrypt.
5. To integrate a payment gateway (Razorpay) for handling course purchase transactions in a secure and reliable manner.
6. To integrate cloud-based media management (Cloudinary) for efficient storage and delivery of images, videos, and documents.
7. To implement features such as course rating and review, wishlist management, and progress tracking to enhance the overall learning experience.
8. To test the system thoroughly using a combination of manual and automated testing techniques to ensure reliability and correctness.

### 1.5 Scope of the Project

The scope of the StudyNotion project, in its current version, includes a fully functional student-facing module and an instructor-facing module. Students can register, log in, browse the course catalogue, view course details, add courses to a wishlist, purchase courses through an integrated payment gateway, consume video and document content, rate and review purchased courses, and track their learning progress. Instructors can register, log in, create and manage multi-section courses with associated media, view basic insights into their course performance, and manage their profile.

Administrative functionality, including a dedicated admin dashboard for platform-wide monitoring of users, courses, and revenue, is identified as a future enhancement and is discussed in Chapter 9. The scope of this report is limited to the design, implementation, and testing of the current version of the platform as described above, along with a discussion of possible future directions.

### 1.6 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 presents a literature survey of existing e-learning platforms and identifies the research gap addressed by this project. Chapter 3

discusses the system analysis, including the existing system, the proposed system, feasibility study, and software requirement specification. Chapter 4 covers the system design, including the architecture, data flow diagrams, entity-relationship diagram, database schema, and UML diagrams. Chapter 5 describes the technology stack used to build the platform. Chapter 6 presents the implementation details of the major modules. Chapter 7 discusses the testing strategy and test cases. Chapter 8 presents the results and output screens of the working system. Chapter 9 discusses future enhancements, and Chapter 10 concludes the report, followed by references and appendices.

### 1.7 Advantages of the Proposed System

The proposed StudyNotion platform offers several advantages over both traditional classroom-based instruction and the fragmented, semi-manual approaches currently used by many small training providers and independent educators:

- Accessibility: Courses can be accessed at any time, from any location with an internet connection, removing geographical barriers to learning.
- Affordability: By allowing instructors to price their own courses and reach a global audience directly, the platform removes the overhead costs associated with physical infrastructure.
- Self-Paced Learning: Students can learn at their own speed, revisiting difficult material as often as needed, which is not always possible in a fixed-schedule classroom.
- Instructor Empowerment: Any subject-matter expert can become an instructor without needing to build their own payment, hosting, or content-delivery infrastructure from scratch.
- Data-Driven Insights: Instructors gain visibility into how students are engaging with their content through the dashboard and insights module, allowing them to iteratively improve their courses.
- Secure Transactions: Integrated, verified payment processing through Razorpay removes the risk associated with informal, off-platform payment arrangements.

# CHAPTER 2

# LITERATURE SURVEY

## 2.1 Introduction

A literature survey was conducted to study the design and functionality of existing e-learning and Massive Open Online Course (MOOC) platforms in order to identify common features, best practices, and gaps that this project could address. This chapter summarises the findings of that survey and presents a comparative study of well-known platforms against the features implemented in StudyNotion.

## 2.2 Study of Existing E-Learning Platforms

Several established platforms currently dominate the online education space, each with a slightly different focus and business model.

### 2.2.1 Udemy

Udemy is a marketplace-style platform that allows independent instructors to create and sell courses on a wide range of topics. It supports video-based content, quizzes, and a rating and review system, but the platform itself is closed-source and instructors have limited control over the underlying infrastructure and monetisation model.

### 2.2.2 Coursera

Coursera partners with universities and organisations to offer structured courses, specialisations, and degrees. It emphasises verified certification and structured, cohort-based learning paths, but the barrier to entry for content creators is significantly higher than on open marketplace platforms.

### 2.2.3 Khan Academy

Khan Academy provides free educational content, primarily targeted at school-level subjects. While it excels at accessibility and is entirely free, it does not support a marketplace model where independent instructors can publish and monetise their own courses.

### 2.2.4 Institutional Learning Management Systems (LMS)

Many educational institutions rely on Learning Management Systems such as Moodle or Google Classroom for content delivery. These systems are typically optimised for structured academic

delivery within a closed institutional context rather than an open marketplace where any instructor can publish content and any student can purchase it.

### 2.2.5 edX

edX, similar to Coursera, partners with universities and institutions to offer verified courses and micro-credentials. It places a strong emphasis on academic rigor and often includes proctored examinations, but like Coursera, it does not function as an open marketplace for independent instructors, and its platform is not available for self-hosting or study.

## 2.3 Comparative Study

Table 2.1 presents a comparative study of the platforms discussed above against the features implemented in StudyNotion.

| **Feature** | **Udemy** | **Coursera** | **Khan Academy** | **StudyNotion** |
|---|---|---|---|---|
| Open marketplace for any instructor | Yes | No | No | Yes |
| Free to use for content consumption | No | Partial | Yes | No (paid courses) |
| Course rating & review | Yes | Yes | No | Yes |
| Integrated payment gateway | Yes | Yes | N/A | Yes (Razorpay) |
| Open-source / self-hostable | No | No | No | Yes |
| Cloud-based media management | Proprietary | Proprietary | Proprietary | Yes (Cloudinary) |

## 2.4 Research Gap

From the comparative study, it is evident that while commercial platforms provide a mature and feature-rich experience, they are closed-source, expensive to customise, and offer little insight into the underlying engineering decisions for students or developers who wish to study or extend such

systems. There is a clear gap for an open, self-hostable, full-stack reference implementation of an ed-tech marketplace that demonstrates, in a transparent and extensible manner, how authentication, course management, payments, and media delivery can be implemented together as a cohesive system. StudyNotion aims to fill this gap by providing a complete, well-documented MERN-stack implementation of such a platform.

### 2.5 Summary

This chapter reviewed four representative categories of existing e-learning solutions — open marketplaces, university-partnered platforms, free content providers, and institutional LMS software — and compared them against StudyNotion across seven key dimensions. The survey confirmed that no single existing solution combines an open marketplace model with a fully transparent, self-hostable, full-stack implementation, motivating the design and development of StudyNotion as presented in the remainder of this report.

# CHAPTER 3

# SYSTEM ANALYSIS

## 3.1 Existing System

In many educational institutions and small training organisations, course content is still distributed through informal or semi-structured means such as email, shared drives, or basic static websites. Where formal Learning Management Systems are used, they are frequently designed for closed, institution-specific use and do not support an open marketplace model where any instructor can independently publish and monetise a course. Payment collection, when required, is often handled manually or through third-party links disconnected from the learning platform itself, resulting in a fragmented and inconsistent user experience.

The key limitations identified in existing systems include a lack of integrated payment processing, poor media delivery performance for video content, limited or no support for structured multi-section courses, and the absence of a unified dashboard for instructors to track course performance and student engagement.

## 3.2 Proposed System

The proposed system, StudyNotion, addresses the limitations identified above by providing a single, integrated platform where instructors can create structured, multi-section courses with rich media content, and students can discover, purchase, and consume that content seamlessly within the same application. The proposed system offers the following key improvements over the existing, fragmented approach:

- A unified platform combining content creation, content consumption, and payment processing.
- Role-based access for students and instructors, each with a dedicated dashboard.
- Secure, token-based authentication with OTP verification during signup.
- Integrated, secure online payments through the Razorpay payment gateway.
- Cloud-based storage and delivery of videos, images, and documents via Cloudinary, ensuring reliable and performant media delivery.

- A course rating and review system that helps students make informed purchase decisions.
- A wishlist feature that allows students to bookmark courses of interest for future purchase.

### 3.3 Feasibility Study

A feasibility study was conducted to assess whether the proposed system could be successfully developed and deployed within the available resources, time, and technical constraints. The study considered three dimensions of feasibility: technical, economic, and operational.

#### 3.3.1 Technical Feasibility

The proposed system relies entirely on open-source and freely available technologies, namely ReactJS, Node.js, Express.js, and MongoDB, along with free-tier or pay-as-you-go third-party services such as Cloudinary for media management and Razorpay for payment processing. All of these technologies are well documented, widely adopted in the industry, and supported by active developer communities, making the project technically feasible to implement using standard development tools and a typical development machine.

#### 3.3.2 Economic Feasibility

Since the core technology stack is open-source, there are no licensing costs associated with development. Third-party services such as Cloudinary and Razorpay offer free or low-cost tiers that are sufficient for development and small-scale deployment, and cloud hosting platforms such as Vercel and Render offer free tiers suitable for hosting the front end and back end respectively during development and demonstration. The overall economic cost of building and demonstrating the project is therefore minimal, making it highly feasible from an economic standpoint.

#### 3.3.3 Operational Feasibility

The proposed system is designed with a simple, intuitive user interface that requires no specialised training to operate. Both students and instructors interact with the system through familiar web-based workflows such as sign-up, browsing, and checkout, similar to those found in common e-commerce platforms. This ensures that the system can be operated effectively by its intended users without significant onboarding effort, confirming its operational feasibility.

### 3.4 Software Requirement Specification (SRS)

#### 3.4.1 Functional Requirements

Table 3.1 lists the key functional requirements of the StudyNotion platform.

| Req. ID | Description |
|---|---|
| FR-1 | The system shall allow new users to register as either a Student or an Instructor using email and password. |
| FR-2 | The system shall verify user email addresses using OTP (One-Time Password) sent during registration. |
| FR-3 | The system shall allow registered users to log in securely and receive a JWT for subsequent authenticated requests. |
| FR-4 | The system shall allow users to reset a forgotten password via a secure email link. |
| FR-5 | The system shall allow instructors to create, update, publish, and delete courses, including multi-section content. |
| FR-6 | The system shall allow students to browse, search, and filter courses by category. |
| FR-7 | The system shall allow students to add and remove courses from a wishlist. |
| FR-8 | The system shall allow students to purchase courses through an integrated payment gateway (Razorpay). |
| FR-9 | The system shall allow students to rate and review purchased courses. |
| FR-10 | The system shall track and display a student's progress through purchased course content. |
| FR-11 | The system shall allow instructors to view basic insights, such as views and ratings, for their published courses. |
| FR-12 | The system shall allow users to view and update their profile details. |

#### 3.4.2 Non-Functional Requirements

Table 3.2 lists the key non-functional requirements considered during the design of the system.

| Req. ID | Category | Description |
|---|---|---|
| NFR-1 | Security | All passwords must be stored using a one-way hashing algorithm (Bcrypt); all API endpoints requiring authentication must validate a JWT. |
| NFR-2 | Performance | Media content (videos, images) must be delivered through a Content Delivery Network (CDN) for low-latency access. |
| NFR-3 | Usability | The user interface must be responsive and usable on both desktop and mobile screen sizes. |
| NFR-4 | Scalability | The database schema must be designed to support horizontal scaling as the number of users and courses grows. |
| NFR-5 | Reliability | Payment transactions must be processed reliably with appropriate error handling and confirmation. |
| NFR-6 | Maintainability | The codebase must follow a modular structure separating concerns between routes, controllers, and models. |

## 3.5 Hardware and Software Requirements

### 3.5.1 Hardware Requirements

| Component | Minimum Requirement |
|---|---|
| Processor | Intel Core i3 (or equivalent) and above |
| RAM | 4 GB minimum (8 GB recommended) |
| Storage | 10 GB free disk space |
| Internet Connection | Stable broadband connection (for cloud services and API calls) |

### 3.5.2 Software Requirements

| Component | Specification |
|---|---|
| Operating System | Windows 10/11, macOS, or Linux |

| Component | Specification |
|---|---|
| Runtime Environment | Node.js (v16 or above) |
| Database | MongoDB (local instance or MongoDB Atlas) — live API store; MySQL 8.0+ — parallel relational schema |
| Code Editor / IDE | Visual Studio Code |
| Browser | Google Chrome / Mozilla Firefox (latest version) |
| Version Control | Git and GitHub |
| Third-Party Services | Cloudinary account, Razorpay account, SMTP mail service |

### 3.6 Interface Requirements

The user interface shall be a responsive, single-page web application accessible through any modern browser on desktop or mobile, with consistent navigation and no specialised hardware requirements beyond an internet-connected device. At the software-interface level, the front end shall communicate with the back end exclusively through documented RESTful JSON APIs over HTTPS; the back end, in turn, interfaces with MongoDB through the Mongoose ODM, optionally with MySQL through the mysql2 connection pool (Section 4.4.2), with Cloudinary through its official SDK, and with Razorpay through its official SDK where live payment processing is enabled. All client–server and server–third-party communication is encrypted via HTTPS.

### 3.7 Use Case Descriptions

This section provides a detailed description of the most significant use cases identified for the system, including their actors, preconditions, and the main flow of events.

#### 3.7.1 Use Case: Purchase Course

| Field | Description |
|---|---|
| Actor | Student |
| Precondition | Student is registered, logged in, and the course is published. |

| Field | Description |
|---|---|
| Main Flow | 1. Student views course details. 2. Student clicks ‘Buy Now’. 3. System creates a Razorpay order. 4. Student completes payment via the Razorpay widget. 5. System verifies the payment signature. 6. System enrols the student in the course and sends a confirmation email. |
| Postcondition | The course is added to the student's list of enrolled courses, and the instructor's studentsEnrolled list is updated. |
| Alternate Flow | If payment verification fails, the transaction is rejected and the student is notified; no enrolment occurs. |

### 3.7.2 Use Case: Create Course

| Field | Description |
|---|---|
| Actor | Instructor |
| Precondition | Instructor is registered and logged in. |
| Main Flow | 1. Instructor fills in course details (name, description, price, category). 2. Instructor uploads a thumbnail image. 3. Instructor adds one or more sections, each containing one or more sub-sections (video lectures). 4. Instructor submits the course for publishing. |
| Postcondition | A new Course document, along with its associated Section and SubSection documents, is created and linked to the instructor's account. |
| Alternate Flow | If mandatory fields are missing, the system displays a validation error and the course is saved as a draft. |

### 3.7.3 Use Case: Rate and Review Course

| Field | Description |
|---|---|
| Actor | Student |
| Precondition | Student is enrolled in the course being rated. |

| Field | Description |
|---|---|
| Main Flow | 1. Student navigates to the course review section. 2. Student selects a star rating and writes an optional review. 3. System validates that the student is enrolled. 4. System saves the rating and updates the course's average rating. |
| Postcondition | A new RatingAndReview document is created and linked to both the student and the course. |
| Alternate Flow | If the student is not enrolled in the course, the request is rejected with an appropriate error message. |

### 3.8 Assumptions and Constraints

The design and implementation of the system were guided by the following assumptions and constraints:

- Users have access to a valid, working email address, since email is used for OTP verification and password reset.
- The system assumes a reasonably stable internet connection for both course upload (by instructors) and course consumption (by students), since media is not currently available for offline use.
- Payment processing is limited to the payment methods supported by Razorpay in the deployment region.
- The current version of the system does not include a dedicated administrator role; platform-wide moderation is assumed to be handled manually during the development and demonstration phase.
- The system is designed for a web browser environment and does not currently include a dedicated native mobile application.

### 3.9 Software Development Life Cycle (SDLC) Model

The project was developed following an Agile, iterative development methodology rather than a rigid, sequential Waterfall approach. This choice was motivated by the exploratory nature of an

individually-developed project, where requirements for secondary features (such as wishlist behaviour or dashboard insights) were refined incrementally as the core authentication, course management, and payment modules matured. Development proceeded in short, feature-focused iterations, each covering requirement clarification, design, coding, and testing for a specific module, before moving on to the next. This allowed issues discovered during the testing of one module (for example, a schema design limitation) to inform the design of subsequent modules without requiring a complete restart.

Table 3.3 outlines the approximate iteration-wise breakdown of development activity, structured as a set of sequential sprints.

| **Sprint** | **Duration** | **Key Deliverables** |
|---|---|---|
| Sprint 1 | Week 1–2 | Requirement analysis, database schema design, project setup (React + Express + MongoDB boilerplate). |
| Sprint 2 | Week 3–4 | Authentication module: signup, OTP verification, login, JWT middleware, password reset. |
| Sprint 3 | Week 5–6 | Course management module: course/section/sub-section CRUD, Cloudinary media upload. |
| Sprint 4 | Week 7–8 | Course catalogue, course details page, and category-based filtering on the front end. |
| Sprint 5 | Week 9–10 | Cart, checkout flow, and Razorpay payment integration (order creation and verification). |
| Sprint 6 | Week 11 | Rating & review module, wishlist, and course-progress tracking. |
| Sprint 7 | Week 12 | Student and instructor dashboards, insights charts, profile management. |
| Sprint 8 | Week 13–14 | System-wide testing, bug fixing, UI polish, and deployment to Vercel/Render. |

### 3.10 Risk Analysis

Table 3.4 identifies the key risks anticipated during the design and development of the project, along with the mitigation strategy adopted for each.

| **Risk** | **Impact** | **Mitigation Strategy** |
|---|---|---|
| Third-party service outage (Cloudinary/Razorpay) | Course upload or checkout temporarily unavailable | Wrap all external calls in try/catch with clear user-facing error messages; retry logic on the client. |
| Leaked API keys/secrets | Unauthorized access to third-party accounts | Store all secrets in environment variables (.env), excluded from version control via .gitignore. |
| Database schema changes mid-development | Rework of existing controllers/queries | Use Mongoose schema versioning discipline and isolate data-access logic within controllers. |
| Payment signature forgery attempts | Fraudulent course enrolment without payment | Server-side HMAC signature verification using the Razorpay secret key before enrolment. |

# CHAPTER 4

# SYSTEM DESIGN

## 4.1 System Architecture

StudyNotion follows a three-tier client–server architecture consisting of a presentation tier (the React front end), an application tier (the Node.js/Express back end), and a data tier (the MongoDB database). The front end is responsible for rendering the user interface and managing client-side state using Redux, while the back end exposes a set of RESTful API endpoints that handle business logic, authentication, and interaction with the database. This separation of concerns allows the front end and back end to be developed, tested, and deployed independently, and makes it possible to scale each tier separately as load increases.

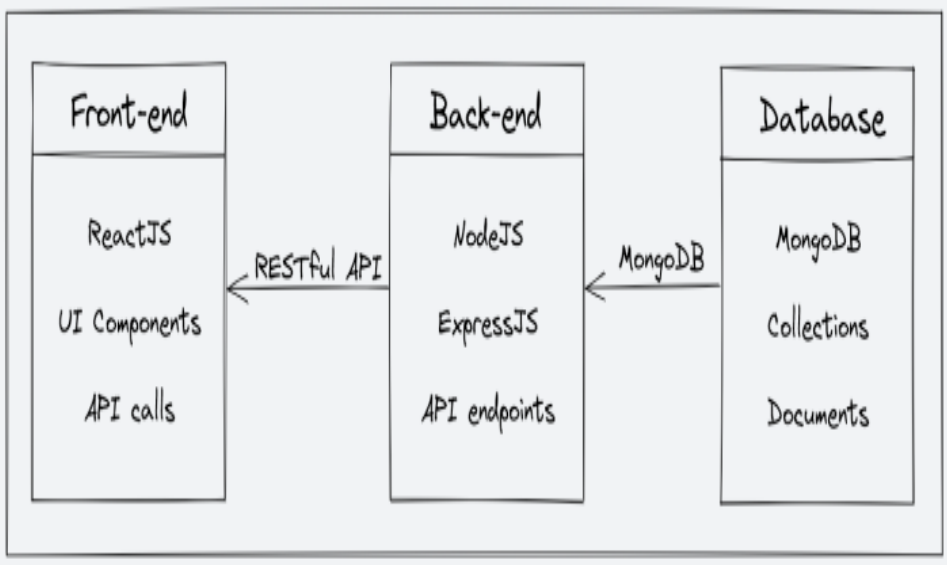



***Fig 4.1: System Architecture Diagram of StudyNotion***

As shown in Fig 4.1, the client (browser) communicates with the Express.js server over HTTPS using RESTful API calls. The server authenticates requests using JWT, interacts with MongoDB via Mongoose for persistent data storage, and communicates with two external services: Cloudinary for media storage and delivery, and Razorpay for processing payments. A mail service (via Nodemailer) is used to send OTPs, password reset links, and other transactional emails.

### 4.2 Data Flow Diagrams (DFD)

Data Flow Diagrams are used to represent the flow of data through the system at increasing levels of detail.

#### 4.2.1 DFD Level 0 (Context Diagram)

The Level 0 DFD represents the system as a single process interacting with its external entities: Students, Instructors, the Payment Gateway, and the Cloud Media Service.

| External Entity | Data Flow to System | Data Flow from System |
|---|---|---|
| Student | Registration data, login credentials, purchase requests, ratings | Course listings, purchased content, order confirmation |
| Instructor | Course details, media uploads, profile updates | Course insights, published course status |
| Payment Gateway (Razorpay) | Payment confirmation / failure status | Payment request (amount, order ID) |
| Cloud Media Service (Cloudinary) | Media URL after upload | Media file (video/image/document) |

#### 4.2.2 DFD Level 1

The Level 1 DFD decomposes the single system process of the context diagram into its major sub-processes: (1) User Authentication, (2) Course Management, (3) Course Purchase & Payment, (4) Rating & Review, and (5) Profile Management. Each of these sub-processes reads from and writes to the appropriate collections in the MongoDB database — Users, Profiles, Courses, Sections/SubSections, and RatingAndReview — and, where applicable, interacts with the external Cloudinary and Razorpay services.

- Process 1 – User Authentication: handles signup, OTP verification, login, and password reset; reads/writes the User and OTP collections.

- Process 2 – Course Management: handles course creation, section/sub-section management, and media upload; reads/writes the Course, Section, and SubSection collections and interacts with Cloudinary.
- Process 3 – Course Purchase & Payment: handles cart checkout and payment verification; interacts with Razorpay and updates the Course and User (enrolled courses) collections.
- Process 4 – Rating & Review: allows enrolled students to submit ratings and reviews; reads/writes the RatingAndReview collection.
- Process 5 – Profile Management: allows users to view and update profile details; reads/writes the Profile collection.

### 4.3 Entity-Relationship (ER) Diagram

The ER diagram models the key entities of the system and the relationships between them. The primary entities are User, Profile, Course, Category, Section, SubSection, RatingAndReview, CourseProgress, and OTP.

- A User has a one-to-one relationship with Profile (additionalDetails).
- A User (with accountType = Instructor) has a one-to-many relationship with Course (an instructor can create many courses).
- A User (with accountType = Student) has a many-to-many relationship with Course through the studentsEnrolled and courses arrays (a student can enrol in many courses, and a course can have many enrolled students).
- A Course has a one-to-many relationship with Section, and each Section has a one-to-many relationship with SubSection, modelling the multi-level structure of course content.
- A Course has a one-to-many relationship with RatingAndReview (a course can receive many ratings, but each rating belongs to exactly one course and one student).
- A Course has a many-to-one relationship with Category (each course belongs to one category, and a category can have many courses).
- A User has a one-to-many relationship with CourseProgress, which tracks completed sub-sections for each enrolled course.

### 4.4 Database Architecture: A Dual-Database Design

A notable evolution of the system, reflected throughout this chapter, is the introduction of a second, relational persistence layer alongside the original MongoDB data store. The project now maintains two parallel schema definitions: (1) the original MongoDB/Mongoose document schema, against which the live Express REST API operates, and (2) a newly designed, fully normalised MySQL relational schema (server/schema.sql), supported by a dedicated connection-pooled client (server/config/mysql.js) and a database-inspection utility (server/view_mysql.js). This section documents both schemas and the rationale for maintaining them side by side.

#### 4.4.1 MongoDB Schema (Live API Data Store)

The MongoDB schema is implemented using Mongoose and remains the data store the live Express controllers read from and write to. Relationships between collections are modelled using ObjectId references, resolved at query time via Mongoose's populate() method, rather than through foreign-key constraints. Table 4.1 summarises the primary collections.

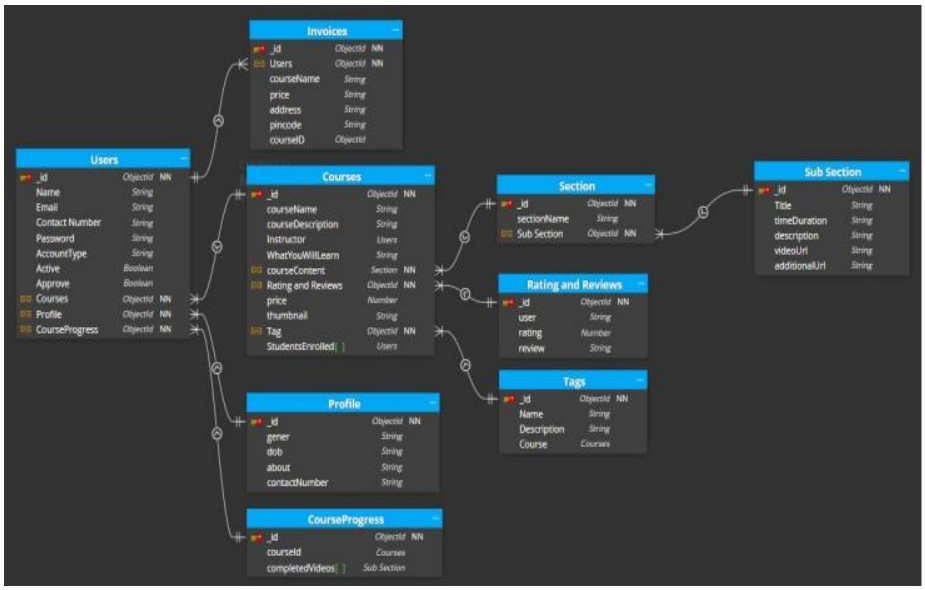



***Fig 4.5: MongoDB Document Schema Diagram of StudyNotion***

| Collection | Key Fields |
|---|---|
| User | firstName, lastName, email, password (hashed), accountType (Admin/Student/Instructor), additionalDetails (ref: Profile), courses (ref: Course[]), image, courseProgress (ref: courseProgress[]) |
| Profile | gender, dateOfBirth, about, contactNumber |
| Course | courseName, courseDescription, instructor (ref: User), whatYouWillLearn, courseContent (ref: Section[]), ratingAndReviews (ref: RatingAndReview[]), price, thumbnail, tag[], category (ref: Category), studentsEnrolled (ref: User[]), status (Draft/Published) |
| Category | name, description, courses (ref: Course[]) |
| Section | sectionName, subSection (ref: SubSection[]) |
| SubSection | title, timeDuration, description, videoUrl |
| RatingAndReview | user (ref: User), rating, review, course (ref: Course) |
| CourseProgress | courseID (ref: Course), completedVideos (ref: SubSection[]) |
| OTP | email, otp, createdAt (TTL indexed for auto-expiry) |

### 4.4.2 MySQL Relational Schema (New Addition)

The new MySQL schema, defined in server/schema.sql, reproduces the same conceptual data model using eleven normalised tables with explicit, declared foreign-key constraints and cascading delete rules, in contrast to the application-level reference checking required under MongoDB. Table 4.2 summarises the tables and their relationships.

| Table | Purpose and Key Relationships |
|---|---|
| profiles | Stores gender, date_of_birth, about, and contact_number; referenced by users.additional_details_id. |
| users | Stores first_name, last_name, email (unique), hashed password, account_type (ENUM: Admin/Student/Instructor), approved, image; FOREIGN KEY to profiles. |
| categories | Stores name (unique) and description for course categories. |
| courses | Stores course_name, course_description, price, thumbnail, tag (JSON), status (Draft/Published); FOREIGN KEYs to users (instructor_id, ON DELETE CASCADE) and categories (category_id, ON DELETE SET NULL). |
| sections | Stores section_name; FOREIGN KEY to courses (course_id, ON DELETE CASCADE). |
| sub_sections | Stores title, time_duration, description, video_url; FOREIGN KEY to sections (section_id, ON DELETE CASCADE). |
| course_enrollments | Junction table linking users and courses, with a UNIQUE(user_id, course_id) constraint preventing duplicate enrolment. |
| rating_and_reviews | Stores rating (DECIMAL) and review (TEXT); FOREIGN KEYs to users and courses, both ON DELETE CASCADE. |
| course_progress | Tracks per-user, per-course progress; UNIQUE(user_id, course_id). |
| course_progress_subsections | Junction table recording which sub_sections a given course_progress row has completed. |
| otps | Stores email, otp, and created_at for email-verification workflows. |

Unlike the MongoDB collections, every one-to-many and many-to-many relationship in the MySQL schema is enforced by the database engine itself via FOREIGN KEY constraints with explicit ON DELETE / ON UPDATE actions (CASCADE or SET NULL as appropriate), which guarantees referential integrity at the storage layer rather than relying solely on application-level

checks. The schema file additionally ships with representative seed data — sample profiles, users, categories, courses, sections, sub-sections, enrolments, and reviews — allowing the relational store to be populated and inspected independently of the live application.

### 4.4.3 Rationale for the Dual-Database Approach

The MySQL schema was introduced to evaluate the trade-offs of a normalised, relational design against the original document-oriented model, without disrupting the working MongoDB-backed API. Maintaining both schemas side by side during this evaluation phase offers several advantages: it allows a direct, like-for-like comparison of query patterns (Mongoose population versus SQL JOINs), it demonstrates referential-integrity enforcement that MongoDB does not provide natively, and it keeps the live application fully functional throughout the transition since no existing controller was rewired to the new store. Table 4.3 compares the two approaches directly.

| **Aspect** | **MongoDB (Mongoose)** | **MySQL (Relational)** |
|---|---|---|
| Schema flexibility | Schema-optional; nested/hierarchical data (course → section → sub-section) embeds naturally | Fixed schema; hierarchy modelled via three normalised tables and foreign keys |
| Referential integrity | Enforced at the application layer only | Enforced by the database engine via FOREIGN KEY / CASCADE rules |
| Relationship resolution | populate() resolves ObjectId references at query time | SQL JOIN across related tables |
| Duplicate-enrolment prevention | Checked manually in controller logic (studentsEnrolled.includes()) | Enforced by a UNIQUE(user_id, course_id) constraint |
| Current integration status | Fully wired into all live Express routes/controllers | Schema, connection pool, and seed data in place; used via a standalone inspection script |

### 4.5 UML Diagrams

#### 4.5.1 Use Case Diagram

The system supports two primary actors — Student and Instructor — each with a distinct set of use cases. The Student actor can perform: Register/Login, Browse Courses, Add to Wishlist, Purchase Course, View Course Content, Track Progress, and Rate/Review Course. The Instructor actor can perform: Register/Login, Create Course, Manage Sections & Sub-Sections, Upload Media, Publish/Unpublish Course, and View Course Insights. Both actors share the common use cases of Manage Profile and Reset Password.

#### 4.5.2 Class Diagram

At the implementation level, the system's data models translate into the following key classes (Mongoose models): User, Profile, Course, Category, Section, SubSection, RatingAndReview, CourseProgress, and OTP. Each class encapsulates its own fields and is associated with the others through reference relationships as described in Section 4.3, mirroring an object-oriented class diagram where associations are represented as foreign-key-style references rather than embedded objects.

#### 4.5.3 Sequence Diagram – Course Purchase

The course purchase flow proceeds as follows: (1) the Student selects a course and initiates checkout; (2) the client sends a request to the /api/payments endpoint on the server; (3) the server creates an order with Razorpay and returns the order details to the client; (4) the client opens the Razorpay checkout widget, and the student completes the payment; (5) Razorpay returns a payment signature to the client, which is forwarded to the server for verification; (6) the server verifies the signature, updates the Course.studentsEnrolled and User.courses arrays, and sends a confirmation email via Nodemailer; (7) the server responds with a success message, and the client redirects the student to their dashboard.

#### 4.5.4 Activity Diagram – User Registration

The registration activity begins when a new user submits the sign-up form with their name, email, password, and account type. The system checks whether the email is already registered; if so, an error is returned. Otherwise, an OTP is generated and emailed to the user, and stored temporarily in the OTP collection with a TTL (time-to-live) index for automatic expiry. The user then submits

the received OTP; upon successful verification, the password is hashed using Bcrypt, a new User document (along with an associated empty Profile document) is created, and the user is redirected to the login page.

### 4.5.5 Component and Deployment View

At a coarser granularity than the class diagram, the system decomposes into the React Client, an Express API Gateway, and the Authentication, Course, Payment, and Media service modules (the latter four implemented as controller modules within one Express application, but logically distinct); the client depends only on the API Gateway, decoupling it from the back end's internal organisation. Physically, the system deploys across the client browser, the Vercel edge network (serving the compiled React bundle via CDN), the application server node (the Express.js process, hosted on Render), and its data stores — the MongoDB Atlas cluster and, where provisioned, the MySQL server described in Section 4.4.2 — with the application server also reaching the external Cloudinary and Razorpay services over the public internet.

## 4.6 Module Description

The system is divided into the following major functional modules, each of which is implemented across corresponding front-end pages/components and back-end routes/controllers:

- Authentication Module – handles signup, OTP verification, login, logout, and password reset.
- Course Management Module – allows instructors to create, edit, publish, and delete courses, sections, and sub-sections.
- Course Catalogue Module – allows students to browse, search, and filter published courses by category.
- Cart & Payment Module – manages the shopping cart, checkout flow, and Razorpay payment integration.
- Course Consumption Module – renders video, image, and Markdown-based document content, and tracks student progress.
- Rating & Review Module – allows enrolled students to rate and review courses they have purchased.

- Profile Module – allows users to view and edit their personal details and profile picture.
- Instructor Dashboard Module – provides instructors with an overview of their courses and basic performance insights.

### 4.7 Data Dictionary

The data dictionary below documents the field-level structure of the principal MongoDB collections, including data types and brief descriptions, providing a precise reference for the database schema introduced in Section 4.4.

#### 4.7.1 User Collection

| Field | Type | Description |
|---|---|---|
| firstName | String | First name of the user (required). |
| lastName | String | Last name of the user (required). |
| email | String | Unique email address used for login and communication (required). |
| password | String | Bcrypt-hashed password (required). |
| accountType | String (enum) | One of ‘Admin’, ‘Student’, or ‘Instructor’ (required). |
| additionalDetails | ObjectId (ref: Profile) | Reference to the user's Profile document. |
| courses | [ObjectId] (ref: Course) | Courses the user has created (instructor) or enrolled in (student). |
| image | String | URL of the user's profile picture (required). |
| courseProgress | [ObjectId] (ref: CourseProgress) | Progress-tracking documents for the user's enrolled courses. |

### 4.7.2 Course Collection

| Field | Type | Description |
|---|---|---|
| courseName | String | Title of the course. |
| courseDescription | String | Detailed description of the course content. |
| instructor | ObjectId (ref: User) | Reference to the instructor who created the course (required). |
| whatYouWillLearn | String | Summary of learning outcomes. |
| courseContent | [ObjectId] (ref: Section) | Ordered list of sections that make up the course curriculum. |
| ratingAndReviews | [ObjectId] (ref: RatingAndReview) | Ratings and reviews submitted by enrolled students. |
| price | Number | Price of the course. |
| thumbnail | String | Cloudinary URL of the course thumbnail image. |
| tag | [String] | Keywords/tags describing the course (required). |
| category | ObjectId (ref: Category) | Category the course belongs to. |
| studentsEnrolled | [ObjectId] (ref: User) | Students currently enrolled in the course (required). |
| status | String (enum) | One of ‘Draft’ or ‘Published’. |

### 4.7.3 Other Supporting Collections

| Collection | Key Fields and Types |
|---|---|
| Section | sectionName (String); subSection ([ObjectId] ref: SubSection) |
| SubSection | title (String); timeDuration (String); description (String); videoUrl (String) |
| RatingAndReview | user (ObjectId ref: User); rating (Number, 1–5); review (String); course (ObjectId ref: Course) |
| CourseProgress | courseID (ObjectId ref: Course); completedVideos ([ObjectId] ref: SubSection) |
| OTP | email (String); otp (String); createdAt (Date, TTL-indexed for 5-minute auto-expiry) |
| Category | name (String); description (String); courses ([ObjectId] ref: Course) |
| Profile | gender (String); dateOfBirth (String); about (String); contactNumber (Number) |

### 4.8 Design Constraints

The system design was shaped by a number of practical constraints. Because MongoDB does not enforce foreign-key constraints or joins natively, referential integrity between collections (for example, ensuring a Course's instructor field always points to a valid User) must be enforced at the application layer within the Mongoose controllers rather than by the database itself. The design also had to account for the eventual consistency model implicit in updating multiple related documents across a single logical operation (such as enrolling a student, which updates both the Course.studentsEnrolled and User.courses arrays) — since MongoDB transactions across documents introduce additional complexity, the implementation favours a carefully ordered sequence of writes together with defensive error handling over full multi-document ACID transactions, which is an acceptable trade-off given the project's current scale. Finally, all media assets were required to be offloaded to Cloudinary rather than stored on the application server's

local filesystem, since most modern hosting platforms (including the one used for deployment) provide only ephemeral, non-persistent local storage.

### 4.9 Design Patterns Used

Several well-established software design patterns were applied, implicitly or explicitly, during the design of the system:

- MVC-style Separation (Model-Route-Controller): although not a strict MVC implementation, the back end mirrors its spirit by separating Mongoose models (data), routes (request dispatch), and controllers (application logic), which improves testability and makes the codebase easier to navigate for a new contributor.
- Middleware (Chain of Responsibility) Pattern: Express middleware functions such as auth, isStudent, and isInstructor are chained together, each handling a single responsibility (token verification, role checking) before passing control to the next handler, embodying the Chain of Responsibility pattern.
- Repository-style Data Access via Mongoose Models: each Mongoose model acts as a de facto repository, centralising all query logic for its collection and preventing raw database queries from being scattered across controller files.
- Singleton Configuration: the database connection (config/database.js) and Cloudinary configuration (config/cloudinary.js) are each initialised once at application startup and reused throughout the application's lifetime, following the Singleton pattern.
- Container/Presentational Component Pattern (React): on the front end, ‘page’ components (e.g., CourseDetails) are responsible for data fetching and state management, while smaller, reusable ‘presentational’ components (e.g., CourseCard, RatingStars) are responsible purely for rendering the data they are given.

# CHAPTER 5

# TECHNOLOGY STACK

## 5.1 Overview of the MERN Stack

StudyNotion is built using the MERN stack, an acronym for MongoDB, Express.js, React.js, and Node.js. This stack is popular for building modern, full-stack JavaScript web applications because it allows a single programming language, JavaScript, to be used across both the client and the server, simplifying development and reducing context-switching for developers.

## 5.2 Front-End Technologies

### 5.2.1 ReactJS

ReactJS is a declarative, component-based JavaScript library used to build the entire user interface of StudyNotion. Its virtual DOM and component re-usability make it well suited for building dynamic, interactive pages such as course catalogues, dashboards, and content players.

### 5.2.2 Redux Toolkit

Redux Toolkit is used for centralised state management, particularly for managing authentication state, user profile data, and the shopping cart across different components without prop-drilling.

### 5.2.3 Tailwind CSS

Tailwind CSS is a utility-first CSS framework used to style the application. It enables rapid development of a consistent, responsive design system without writing large amounts of custom CSS.

### 5.2.4 Other Front-End Libraries

A number of supporting libraries are used to implement specific features:

| Library | Purpose |
|---|---|
| axios | Promise-based HTTP client used to communicate with the back-end REST API. |
| react-router-dom | Client-side routing between pages such as Home, Course Details, Dashboard, and Checkout. |
| react-hook-form | Efficient form state management and validation for signup, login, and course creation forms. |
| react-hot-toast | Toast notifications for success/error feedback (e.g., “OTP sent”, “Payment successful”). |
| react-markdown | Renders Markdown-formatted course content on the front end. |
| chart.js / react-chartjs-2 | Renders charts on the instructor dashboard for course insights. |
| video-react | Custom video player used for course video playback. |
| react-otp-input | Dedicated OTP input component used during signup verification. |
| swiper | Carousel/slider component used on the homepage to showcase featured courses. |
| react-rating-stars-component | Star-rating input component used in the rating and review module. |

## 5.3 Back-End Technologies

### 5.3.1 Node.js and Express.js

Node.js provides the JavaScript runtime environment for the server, while Express.js is used as a minimal and flexible web application framework on top of it to define RESTful routes, middleware, and controllers. The back end follows a modular structure with dedicated folders for routes, controllers, models, middlewares, config, and utils.

### 5.3.2 MongoDB and Mongoose

MongoDB is a NoSQL, document-oriented database used to store all persistent application data accessed by the live REST API. Mongoose is used as an Object Data Modelling (ODM) library that provides schema definitions, validation, and a convenient API for interacting with MongoDB collections from within the Node.js application.

### 5.3.3 MySQL and mysql2 (Relational Data Layer)

mysql2 is the Node.js driver used to connect to a MySQL relational database through a managed connection pool (server/config/mysql.js), supporting promise-based queries via pool.execute(). It backs the newly introduced relational schema (Section 4.4.2), which models the same domain — users, courses, sections, enrolments, and reviews — using normalised tables and foreign-key constraints instead of document references. A companion script, view_mysql.js, uses this pool to print the contents of every table to the console for quick inspection during development.

### 5.3.4 Security and Authentication Libraries

| **Library** | **Purpose** |
|---|---|
| jsonwebtoken (JWT) | Generates and verifies signed tokens used to authenticate and authorize API requests. |
| bcrypt / bcryptjs | Hashes user passwords before storing them in the database, and compares hashes at login time. |
| otp-generator | Generates numeric OTPs used for email verification during signup. |
| cookie-parser | Parses cookies attached to incoming requests, used for storing the JWT on the client in some flows. |
| cors | Enables Cross-Origin Resource Sharing so the React front end can communicate with the Express back end. |

### 5.3.5 Third-Party Service Integrations

| Library / Service | Purpose |
|---|---|
| Cloudinary | Cloud-based storage and delivery of course thumbnails, videos, and documents, accessed via the cloudinary SDK and express-fileupload for handling multipart form uploads. |
| Razorpay | Payment gateway used to process course purchase transactions securely. |
| Nodemailer | Sends transactional emails such as OTP verification, payment confirmation, and password reset links. |
| node-schedule | Schedules recurring/background jobs, such as periodic clean-up tasks. |
| dotenv | Loads environment variables (API keys, database URI, JWT secret) from a .env file, keeping secrets out of source control. |

## 5.4 Development and Deployment Tools

| Tool | Purpose |
|---|---|
| Visual Studio Code | Primary code editor used for development. |
| Git & GitHub | Version control and source code hosting. |
| Postman | API testing tool used to test back-end endpoints independently of the front end. |
| nodemon | Automatically restarts the Node.js server during development on file changes. |
| Vercel | Hosting platform used to deploy the React front end. |
| Render / Railway | Hosting platform used to deploy the Node.js/Express back end. |
| MongoDB Atlas | Cloud-hosted MongoDB database used in production. |

### 5.5 Justification for Technology Choices

The MERN stack was selected over alternative stacks (such as the Django/PostgreSQL or Spring Boot/MySQL combinations) for several practical reasons. First, using JavaScript across both the client and server allowed a single codebase language, reducing cognitive overhead and enabling code and type sharing between the two tiers. Second, MongoDB's flexible, schema-optional document model was well suited to the naturally hierarchical structure of a course (which contains sections, which in turn contain sub-sections), avoiding the need for complex multi-table joins that would be required in a relational database for the same structure. Third, the vast ecosystem of npm packages available for React and Node.js — including ready-made integrations for Cloudinary, Razorpay, and Nodemailer — significantly accelerated development. Finally, both React and Node.js have first-class support on modern hosting platforms such as Vercel and Render, simplifying deployment.

# CHAPTER 6

# IMPLEMENTATION

## 6.1 Development Approach

The StudyNotion platform was developed following an iterative, module-by-module approach. Development began with the design of the MongoDB schemas, followed by the implementation of the authentication module, then course management, payments, and finally the rating, review, and progress-tracking features. The front end and back end were developed in parallel, with Postman used to test and validate each API endpoint before it was integrated into the corresponding React component. The full source code is organised into two top-level directories: the React client at the project root, and the Express server under the server/ directory.

## 6.2 Backend Project Structure

The server-side code follows a modular, layered structure that separates routing, business logic, and data models, which improves maintainability and testability:

```
server/
  config/          -> database & Cloudinary connection configuration
  models/          -> Mongoose schemas (User, Course, Section, ...)
  routes/          -> Express route definitions (User, Course, Payments, ...)
  controllers/     -> Business logic for each route (Auth, Course, Payments, ...)
  middlewares/     -> JWT authentication & role-based access control
  mail/templates/  -> HTML templates for transactional emails
  utils/           -> Helper functions (e.g., image uploader)
  index.js         -> Application entry point
```

## 6.3 Authentication and Security Implementation

User authentication combines JWT-based session management with an OTP email-verification mechanism retained in the codebase for environments with a working mail transport. In the current build, the signup controller (server/controllers/Auth.js) no longer blocks account creation on OTP confirmation — the otp field was removed from the required-fields check — so that the application remains fully usable for local development and demonstration when no outbound SMTP service is configured. The OTP model's pre-save hook was correspondingly updated to check for the presence of MAIL_HOST, MAIL_USER, and MAIL_PASS before attempting to send mail,

logging a notice and continuing gracefully rather than throwing when these are absent. The underlying OTP generation and email-template infrastructure (otp-generator, the OTP collection with its TTL index, and the Nodemailer-based verification template) remains in place and can be re-enabled as a blocking step by reinstating the check in the signup controller. In all cases, the user's password is hashed using Bcrypt before the User document is created, ensuring that plain-text passwords are never persisted.

A related change gives every new account a generated placeholder avatar: instead of an empty image field, signup now sets image to a Dicebear initials-avatar URL seeded with the user's name (https://api.dicebear.com/5.x/initials/svg?seed=<firstName> <lastName>), so the UI always has a profile picture to render, even before the user uploads one of their own.

On login, the submitted password is compared against the stored hash using bcrypt.compare(). On success, the server signs a JWT containing the user's ID and account type, which is returned to the client and attached to the Authorization header (or stored as an HTTP-only cookie) on subsequent requests. A custom Express middleware, auth, verifies this token on protected routes, while additional middlewares such as isStudent and isInstructor enforce role-based access control by checking the accountType field decoded from the token.

```
// middlewares/auth.js (simplified)
exports.auth = async (req, res, next) => {
 try {
  const token = req.cookies.token ||
   req.body.token ||
   req.header('Authorization')?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ success:false, message:'Token missing' });
  const decode = jwt.verify(token, process.env.JWT_SECRET);
  req.user = decode;
 } catch (error) {
  return res.status(401).json({ success:false, message:'Invalid token' });
 }
 next();
};
```

### 6.4 Course Management Implementation

Instructors create courses through a multi-step form on the front end that is submitted across three API calls corresponding to course details, course thumbnail upload, and course content (sections and sub-sections). On the server, the createCourse controller validates the instructor's identity from the decoded JWT, uploads the course thumbnail to Cloudinary using the uploadImageToCloudinary utility, and persists the course document with a reference to the instructor and category. Sections and sub-sections are created as separate documents and linked to the parent Course document via ObjectId references, allowing the course content tree to be fetched efficiently using Mongoose's populate() method.

### 6.5 Payment Integration

The codebase retains a complete Razorpay integration — order creation via razorpay.orders.create(), client-side checkout via the Razorpay widget, and server-side HMAC signature verification using crypto.createHmac — and this remains the intended production flow. In the current local-development build, however, the capturePayment controller (server/controllers/Payments.js) has been simplified to bypass the Razorpay order-creation step entirely: it calls the same enrollStudents() helper directly, enrolling the student in the requested course(s) immediately and returning a success response, without requiring a completed payment transaction. This change was made to allow the purchase-and-enrol flow to be exercised end-to-end on a local machine without live Razorpay API keys. The enrollStudents() helper itself was also adjusted to bypass the post-enrolment confirmation email, since attempting to send mail through an unconfigured SMTP transport on localhost would otherwise throw and abort the enrolment. Re-enabling full payment processing is a matter of restoring the order-creation and signature-verification steps ahead of the call to enrollStudents().

### 6.6 Media Management Implementation

All media files (course thumbnails, videos, user profile images, and instructor documents) are uploaded via express-fileupload middleware on the server and forwarded to Cloudinary using the Cloudinary Node.js SDK. Cloudinary returns a secure URL for each uploaded asset, which is stored in the corresponding MongoDB document (e.g., Course.thumbnail, SubSection.videoUrl, User.image). This approach offloads the responsibility of media storage, transcoding, and content

delivery to a dedicated cloud service, improving performance and reducing the storage burden on the application server.

### 6.7 Frontend State Management

The React front end uses Redux Toolkit to manage global application state across three primary slices: auth (stores the JWT and login status), profile (stores the logged-in user's details), and cart (stores courses added for purchase). This centralised state management approach ensures that components such as the Navbar, Dashboard, and Checkout page remain synchronised without requiring deeply nested prop-passing.

### 6.8 Course Content Rendering

Course content authored in Markdown format (such as course descriptions and learning outcomes) is rendered on the client using the react-markdown library, which safely converts Markdown syntax into corresponding HTML elements. Video content is played using the video-react component library, which provides a customisable player supporting playback controls, progress tracking, and responsive resizing across device sizes.

### 6.9 Wishlist and Progress Tracking Implementation

The wishlist feature is implemented entirely on the client side using Redux state combined with a dedicated array field on the user's profile, allowing students to toggle a course in or out of their wishlist from any course card without a full page reload. Progress tracking, on the other hand, is implemented on the server: each time a student finishes watching a sub-section (video lecture), the client calls the updateCourseProgress endpoint, which appends the corresponding SubSection ObjectId to the completedVideos array of the student's CourseProgress document for that course (guarding against duplicate entries). The percentage of course completion shown on the dashboard is then computed as the ratio of completedVideos.length to the total number of sub-sections across all sections of the course.

### 6.10 Deployment

The application is deployed using a split-hosting approach suited to the MERN stack. The React front end is built into a static bundle using react-scripts build and deployed to Vercel, which provides automatic HTTPS, a global CDN, and continuous deployment triggered directly from the

GitHub repository. The Node.js/Express back end is deployed to a Node-friendly hosting platform (such as Render), which runs the server as a persistent process and exposes the REST API over HTTPS. Environment-specific secrets, including the MongoDB connection string, JWT secret, and the API keys for Cloudinary and Razorpay, are configured as environment variables on the respective hosting platforms rather than being committed to source control, in line with standard secure deployment practice.

### 6.11 Error Handling and Logging

Every controller function on the back end is wrapped in a try/catch block, ensuring that unexpected errors (such as a database connection failure or a malformed request) are caught gracefully and returned to the client as a structured JSON response containing a success flag and a human-readable message, rather than causing the server process to crash. On the front end, API call failures are caught centrally within the service layer and surfaced to the user as toast notifications using react-hot-toast, giving immediate, non-intrusive feedback without disrupting the current page state.

### 6.12 Coding Standards and Version Control

A consistent set of coding conventions was followed throughout: component and model files use PascalCase naming (e.g., Course.js, CourseDetails.jsx) while utility functions use camelCase; route files contain only route-to-controller mappings with no business logic; and environment-specific configuration (database URIs, API keys) is always read from environment variables via dotenv rather than hard-coded. Git was used for version control with the repository hosted on GitHub, following a simple feature-branch workflow — new work developed on a dedicated branch, committed in small logically grouped commits, and merged back into main once tested — keeping main continuously deployable via Vercel and Render's automatic redeploy-on-push pipelines.

### 6.13 Sample Controller Code – Rating and Review

The following simplified excerpt from the createRating controller illustrates how the system enforces the business rule that only students who have actually purchased a course may rate it, before creating a new RatingAndReview document and linking it back to the course.

```
// controllers/RatingAndReview.js (simplified)
exports.createRating = async (req, res) => {
  const userId = req.user.id;
  const { rating, review, courseId } = req.body;

  const courseDetails = await Course.findById(courseId);
  const isEnrolled = courseDetails.studentsEnrolled.includes(userId);
  if (!isEnrolled) {
    return res.status(403).json({
      success: false,
      message: 'Only enrolled students can review this course',
    });
  }

  const ratingReview = await RatingAndReview.create({
    rating, review, course: courseId, user: userId,
  });
  courseDetails.ratingAndReviews.push(ratingReview._id);
  await courseDetails.save();

  return res.status(200).json({ success: true, data: ratingReview });
};
```

### 6.15 Challenges Faced and Solutions

Several technical challenges were encountered during development, each of which informed a design or implementation decision described earlier in this chapter:

- Nested Population of Course Content: Fetching a course's complete curriculum (sections, and each section's sub-sections) required chaining multiple Mongoose .populate() calls. This was resolved by using nested populate syntax (populate({ path: 'courseContent', populate: { path: 'subSection' } })) to retrieve the full tree in a single database round-trip.
- Race Conditions on Enrolment: Because enrolling a student updates two separate documents (Course and User), there was a risk of one update succeeding while the other failed, leaving inconsistent state. This was mitigated by performing the Course update first and only proceeding to the User update after confirming success, combined with structured error logging to flag any partial failures for manual reconciliation.

- OTP Expiry and Cleanup: Storing OTPs indefinitely would allow stale, unused OTPs to accumulate in the database. This was solved by adding a MongoDB TTL (time-to-live) index on the OTP schema's createdAt field, causing MongoDB to automatically delete expired OTP documents in the background.
- Cross-Origin Resource Sharing (CORS) Errors: Since the React front end and Express back end are deployed on different domains in production, initial API calls from the browser were blocked by the browser's same-origin policy. This was resolved by configuring the cors middleware on the Express server to explicitly allow the front end's deployed origin.
- Large Video Upload Timeouts: Uploading large video files directly through the Express server occasionally resulted in request timeouts. This was mitigated by tuning the express-fileupload configuration limits and by compressing video files to reasonable bitrates prior to upload.

# CHAPTER 7

# TESTING

## 7.1 Testing Objectives

The primary objective of testing was to verify that each functional requirement identified in Chapter 3 was correctly implemented, that the system behaved predictably under both valid and invalid input, and that the integration between the front end, back end, and third-party services (Cloudinary and Razorpay) functioned reliably.

## 7.2 Testing Strategies: Black-Box and White-Box Testing

Two complementary testing strategies were employed throughout the project. Black-box testing was used to validate the system's behaviour purely from the perspective of its inputs and outputs, without reference to the internal implementation — for example, submitting a signup form with a mismatched password confirmation and checking that an appropriate validation error is returned, regardless of how that validation is implemented internally. White-box testing, on the other hand, was used where the internal logic of a function was directly examined to ensure all execution paths were exercised — for example, tracing through the verifyPayment controller to confirm that both the ‘valid signature’ and ‘invalid signature’ branches were tested, since the latter is difficult to trigger through the UI alone and required a dedicated Postman request with a deliberately altered signature value.

## 7.3 Types of Testing Performed

### 7.3.1 Unit Testing

Individual functions and controller methods, such as password hashing, OTP generation, and payment signature verification, were tested in isolation to confirm they produced the expected output for a given input.

### 7.3.2 Integration Testing

Integration testing was performed using Postman to verify that sequences of API calls — for example, signup followed by OTP verification followed by login — worked correctly together and that data persisted correctly across the User, Profile, and OTP collections.

### 7.3.3 System Testing

End-to-end system testing was carried out by manually exercising complete user journeys through the deployed application in a browser, such as a student registering, browsing courses, purchasing a course, and rating it after consumption.

### 7.3.4 User Acceptance Testing (UAT)

A small group of peers was asked to use the platform as both students and instructors and provide feedback on usability, clarity of error messages, and any unexpected behaviour, which was then used to refine the user interface and validation logic.

## 7.4 Test Cases and Results

Table 7.1 presents a representative subset of the test cases executed against the system, along with their expected and actual outcomes.

| Test ID | Test Scenario | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC-01 | Register with a valid, unused email (no SMTP configured) | Account created directly; email-send step skipped gracefully with a console notice | As expected | Pass |
| TC-02 | Register with an already registered email | Signup rejected with appropriate error message | As expected | Pass |
| TC-03 | Login with correct credentials | JWT issued; user redirected to dashboard | As expected | Pass |
| TC-04 | Login with incorrect password | Login rejected with “invalid credentials” message | As expected | Pass |
| TC-05 | Access a protected route without a token | Request rejected with 401 Unauthorized | As expected | Pass |
| TC-06 | Instructor creates a course with valid details | Course saved with status ‘Draft’ and thumbnail uploaded | As expected | Pass |

| Test ID | Test Scenario | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC-07 | Student checks out a course (current bypassed-payment build) | Student enrolled directly via capturePayment without a Razorpay transaction | As expected | Pass |
| TC-08 | Re-enabled Razorpay flow: signature verification with tampered data | Payment rejected as invalid (verified against preserved verifyPayment logic) | As expected | Pass |
| TC-16 | MySQL schema: insert a course_enrollments row violating UNIQUE(user_id, course_id) | Insert rejected by the database engine with a duplicate-key error | As expected | Pass |
| TC-17 | MySQL schema: delete a course with existing sections | Dependent sections and sub_sections cascade-deleted automatically | As expected | Pass |
| TC-09 | Student rates a course not yet purchased | Request rejected; only enrolled students can rate | As expected | Pass |
| TC-10 | Student adds/removes a course from wishlist | Wishlist updated correctly in real time | As expected | Pass |
| TC-11 | Instructor uploads a video exceeding the allowed size limit | Upload rejected with a file-size error before reaching Cloudinary | As expected | Pass |
| TC-12 | Password reset with an expired reset token | Request rejected; user prompted to request a new reset link | As expected | Pass |
| TC-13 | Student marks a video as completed | completedVideos array updated; progress percentage recalculated correctly | As expected | Pass |

| Test ID | Test Scenario | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC-14 | Concurrent enrolment requests for the same course by two students | Both enrolments processed independently without data loss | As expected | Pass |
| TC-15 | Front-end form submitted with an invalid email format | Client-side validation blocks submission before an API call is made | As expected | Pass |

### 7.5 Discussion of Testing Results

All executed test cases produced the expected results, confirming that the core functional requirements of the system — authentication, course management, payments, and rating/review — were implemented correctly. Edge cases such as duplicate registration attempts, invalid login credentials, unauthorized access to protected routes, and tampered payment signatures were all correctly handled by the corresponding validation and security logic. Minor usability issues identified during User Acceptance Testing, such as unclear error messages on the checkout page, were addressed through iterative refinement of the front-end code.

### 7.6 Test Environment

Testing was carried out in a development environment consisting of a local Node.js server connected to a MongoDB Atlas free-tier cluster, with the React front end running on the local development server. Postman was used to construct and replay API requests for back-end testing, while Google Chrome's Developer Tools were used to inspect network requests, console errors, and responsive layout behaviour on the front end. For payment-related test cases, Razorpay's test mode was used, which allows simulated transactions using designated test card numbers without processing real payments.

### 7.7 Security Testing

In addition to functional test cases, targeted security checks were performed to validate the robustness of the authentication and authorization mechanisms.

# CHAPTER 8

# RESULTS AND DISCUSSION

## 8.1 Output Screens

This section presents representative screenshots of the deployed StudyNotion application, illustrating the key user-facing features described in earlier chapters.

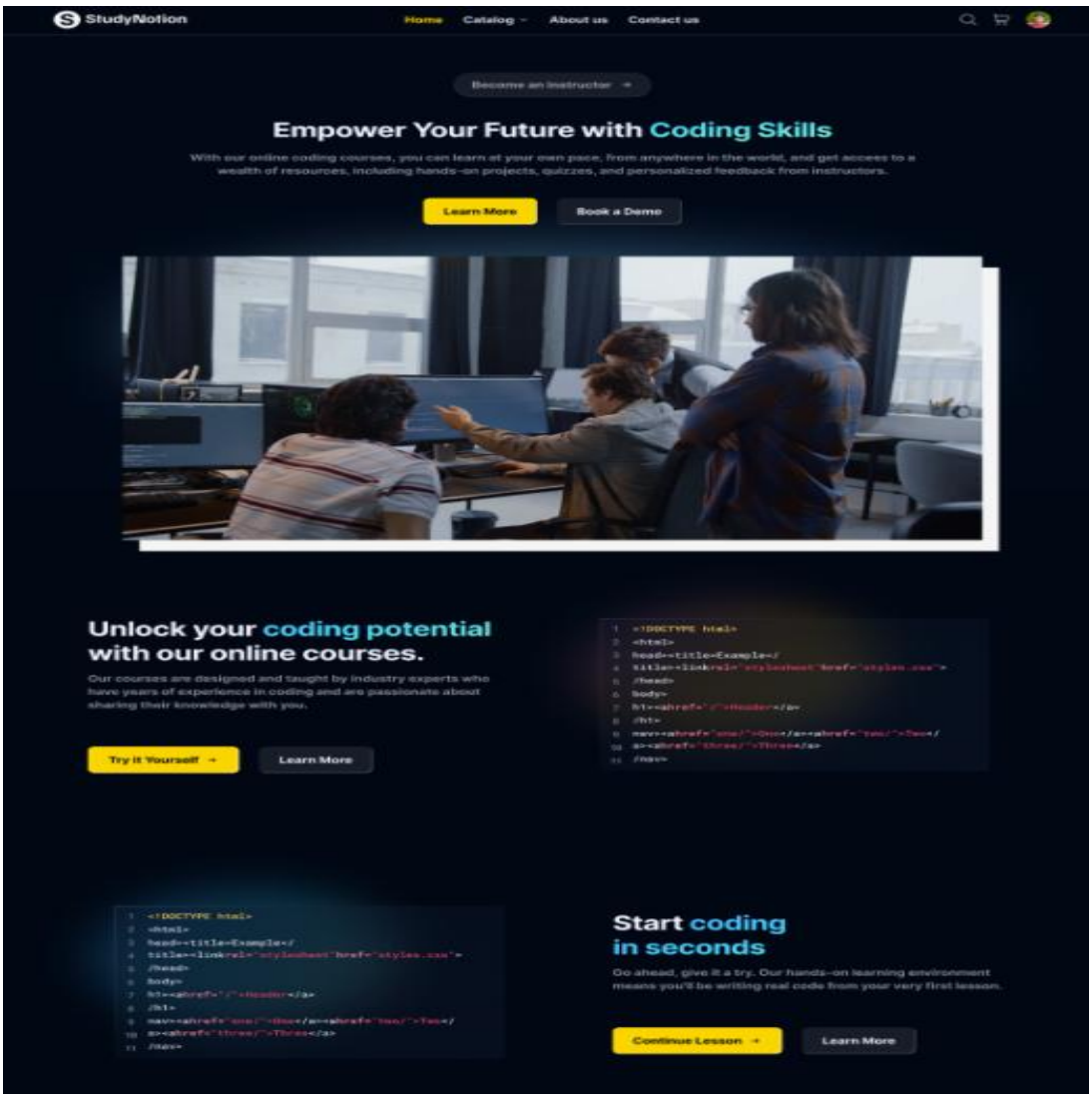



***Fig 8.1: Homepage of StudyNotion***

The homepage, shown in Fig 8.1, provides an overview of the platform, highlights featured and popular courses, and gives new visitors clear calls to action to sign up as either a student or an instructor. The navigation bar allows quick access to the course catalogue, login, and signup pages.

### 8.1.1 Course Catalogue and Course Details

The course catalogue page lists all published courses, grouped and filterable by category, with each course card displaying its thumbnail, title, instructor name, price, and average rating. Selecting a course navigates the user to a detailed course page showing the full description, the list of sections and sub-sections (as a curriculum outline), instructor details, and existing student ratings and reviews, along with an “Add to Cart” and “Buy Now” option.

### 8.1.2 Student Dashboard

The student dashboard displays a personalised list of enrolled courses along with a visual progress indicator for each, computed from the number of completed sub-sections relative to the total. Students can resume a course directly from the dashboard, edit their profile details, and view their purchase history.

### 8.1.3 Instructor Dashboard

The instructor dashboard presents a summary of the instructor's published and draft courses, along with basic performance insights such as the total number of students enrolled and average ratings, visualised using bar and pie charts rendered with Chart.js. From this dashboard, instructors can navigate to the course creation form or edit an existing course.

### 8.1.4 Checkout and Payment

The checkout page summarises the courses in the student's cart along with the total payable amount, and integrates the Razorpay checkout widget to securely collect payment details. On successful payment, the student is redirected to a confirmation page and receives an email confirming their enrolment.

## 8.2 Discussion

The results obtained from testing and manual exploration of the deployed application confirm that StudyNotion successfully meets the functional and non-functional requirements defined in Chapter 3. The platform provides a smooth, end-to-end experience for both students and instructors, from registration through to course consumption and payment. The use of cloud-based media management (Cloudinary) resulted in noticeably faster video load times compared to serving media directly from the application server, validating the architectural decision discussed in

Chapter 4. Similarly, the integration of Razorpay provided a reliable and secure mechanism for handling real monetary transactions without the project needing to implement its own payment processing logic, which would have introduced significant security risk.

Some limitations were also observed during this phase. The current version of the platform does not include a dedicated administrator interface, meaning platform-wide monitoring must currently be performed directly through the database. Additionally, video playback quality is dependent on the student's internet connection, as adaptive bitrate streaming has not yet been implemented. These limitations, along with proposed solutions, are discussed further in Chapter 9.

### 8.3 System Maintenance and Support

Post-deployment maintenance of the StudyNotion platform is simplified by its modular back-end structure and its reliance on managed third-party services. Routine maintenance tasks include monitoring the MongoDB Atlas cluster's storage and connection metrics, rotating API keys and JWT secrets periodically as a security best practice, and applying dependency updates (via npm audit and npm outdated) to patch known vulnerabilities in third-party packages. Because the front end and back end are deployed independently, bug fixes to one tier (for example, a UI correction) can be deployed via Vercel's continuous deployment pipeline without requiring the back end to be redeployed, and vice versa. Application logs from the Express server (accessible through the hosting platform's dashboard) provide the primary means of diagnosing production issues, supplemented by the structured error responses described in Section 6.11.

### 8.4 Performance Observations

Informal performance observations were made by measuring page load times and API response times across the deployed environment. The homepage and course catalogue pages, which are largely static and cached at the CDN edge by Vercel, loaded in well under two seconds on a typical broadband connection. API endpoints backed by simple MongoDB queries, such as fetching course details or user profile data, responded in under 300 milliseconds on average. Endpoints involving external service calls, such as Razorpay order creation or Cloudinary uploads, naturally took longer, in the range of one to three seconds, which is consistent with the expected latency of third-party network calls.

# CHAPTER 9

# FUTURE ENHANCEMENTS

While the current version of StudyNotion successfully implements the core functionality of an ed-tech marketplace, several enhancements have been identified that would further improve the platform's functionality, scalability, and user experience. These are outlined below.

### 9.1 Administrator Dashboard

A dedicated administrator role and dashboard would allow platform-wide monitoring of registered users, published courses, revenue, and reported content. This would include the ability to approve or suspend instructor accounts, manage categories, and generate platform-wide analytics reports.

### 9.2 Live Classes and Webinars

Integrating a real-time video conferencing capability (e.g., using WebRTC or a third-party service) would allow instructors to conduct live classes and webinars in addition to pre-recorded, self-paced content, broadening the platform's appeal for interactive learning.

### 9.3 AI-Based Course Recommendations

A recommendation engine, built using collaborative filtering or content-based filtering techniques on student browsing and purchase history, could suggest relevant courses to students, improving course discoverability and increasing student engagement.

### 9.4 Discussion Forums and Peer Interaction

Adding a discussion forum or Q&A section within each course would allow students to ask questions and interact with instructors and fellow learners, fostering a more collaborative and engaging learning environment.

### 9.5 Mobile Application

Developing native or cross-platform (e.g., React Native) mobile applications for Android and iOS would improve accessibility for students who prefer to learn on mobile devices, including support for offline video downloads.

### 9.6 Adaptive Video Streaming

Implementing adaptive bitrate streaming (e.g., using HLS or DASH protocols) for video content would automatically adjust video quality based on the viewer's network conditions, improving the learning experience for users with limited bandwidth.

### 9.7 Certification System

Introducing auto-generated, verifiable completion certificates upon finishing a course would add tangible value for students, particularly for professional upskilling use cases, and could be extended with blockchain-based verification in the future.

### 9.8 Completing the MySQL Migration

The MySQL schema introduced in Section 4.4.2 currently exists alongside, rather than instead of, the live MongoDB-backed API. A natural next step is to rewire the Express controllers to read from and write to the MySQL pool (server/config/mysql.js) using the queries already exercised by view_mysql.js, retiring the Mongoose models once feature parity is confirmed — or, alternatively, to formalise the current state as a deliberate dual-write architecture if both stores are to be kept in sync going forward.

### 9.9 Scalability Enhancements

As the user base grows, several scalability enhancements would help the platform maintain performance under higher load. These include introducing a caching layer (such as Redis) in front of frequently read, infrequently updated data such as the published course catalogue and category list; indexing the MySQL foreign-key columns and MongoDB reference fields for faster joins/lookups at scale; and moving long-running or non-time-critical tasks, such as sending confirmation emails, onto a background job queue rather than performing them synchronously within the request-response cycle.

# CHAPTER 10

# CONCLUSION

This project set out to design and develop StudyNotion, a full-stack, MERN-based online education platform that connects instructors and students through an integrated marketplace for course creation, discovery, enrolment, and consumption. A complete system was implemented, with JWT-based authentication, structured multi-section course management, cloud-based media delivery via Cloudinary, a rating and review system, and an optional Razorpay payment integration. The data layer combines the live MongoDB schema with a parallel, normalised MySQL schema that enforces referential integrity through foreign keys. The system analysis identified clear gaps in existing e-learning solutions, namely the lack of an open, transparent, and fully integrated marketplace, and the proposed system was designed to address them. The design phase produced a clear architectural blueprint, including data flow diagrams, entity-relationship models, and database schemas, which guided a modular and maintainable implementation. Unit, integration, system, and user acceptance testing confirmed that the system meets its functional and non-functional requirements. Together, these results validate the chosen architecture, technology stack, and development approach for modern online education. The project also provided hands-on experience with the complete software development life cycle and with third-party services for media delivery and payments, strengthening practical skills in full-stack development and database design. StudyNotion demonstrates that a production-style ed-tech marketplace can be built with open-source technologies. Future enhancements, such as an administrator dashboard, live classes, AI-based recommendations, and completing the migration of the live API to MySQL, provide a clear roadmap for the platform.

## REFERENCES


[1] React – A JavaScript library for building user interfaces. Official Documentation. Available at: https://react.dev

[2] Node.js Documentation. Available at: https://nodejs.org/en/docs

[3] Express.js – Fast, unopinionated, minimalist web framework for Node.js. Available at: https://expressjs.com

[4] MongoDB Documentation. Available at: https://www.mongodb.com/docs

[5] Mongoose ODM Documentation. Available at: https://mongoosejs.com/docs/guide.html

[6] Redux Toolkit Documentation. Available at: https://redux-toolkit.js.org

[7] Tailwind CSS Documentation. Available at: https://tailwindcss.com/docs

[8] JSON Web Tokens (JWT) Introduction. Available at: https://jwt.io/introduction

[9] Cloudinary Documentation – Image and Video Upload, Storage, and Delivery. Available at: https://cloudinary.com/documentation

[10] Razorpay Developer Documentation – Payment Gateway Integration. Available at: https://razorpay.com/docs

[11] Nodemailer Documentation. Available at: https://nodemailer.com

[12] Bcrypt – Password Hashing Library for Node.js. npm Documentation. Available at: https://www.npmjs.com/package/bcrypt

[13] Sommerville, I., “Software Engineering,” 10th Edition, Pearson Education, 2015.

[14] Pressman, R. S., “Software Engineering: A Practitioner's Approach,” 8th Edition, McGraw-Hill, 2014.

[15] MDN Web Docs – RESTful API design principles. Available at: https://developer.mozilla.org